

Deep Learning for Visual Computing 2020, Assignment 2

Dominik Schörkhuber
1027470

Thomas Kaufmann
1129115

May 27, 2020

1 Part 1 - Optimizers

In the first part of this assignment we are familiarizing with different optimization algorithms. In Deep Learning we use optimization algorithms, to train the parameters of a neural network. A error (or loss-) function is formulated based on the input and expected output of a network, the optimizer is used to adapt the model parameters such that the loss function is minimized. While Deep Learning problems can typically grow to millions of parameters, we observe in our toy example a 2-dimensional problem space. One can interpret this problem as finding the lowest position on a height map. We visualize the problem space with false color representation, where red areas and blue areas correspond to high values and low values, respectively.

1.1 Stochastic Gradient Descent

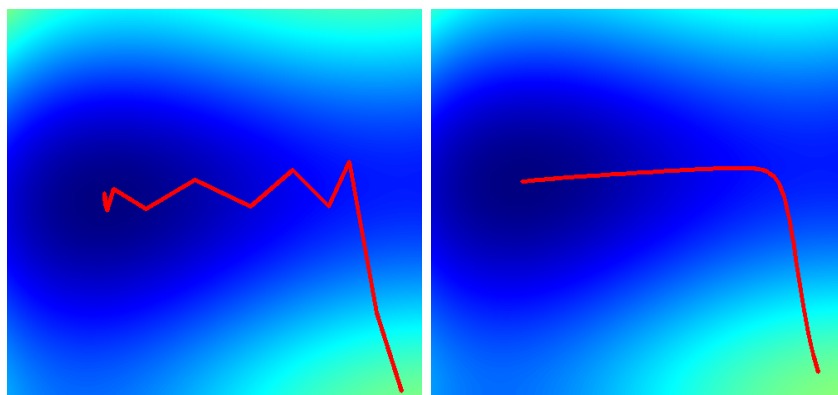
We start our investigation with the Stochastic Gradient Descent (SGD) algorithm. Gradient descent refers to stepping along the steepest slope. This idea is the basis for all of the covered optimization techniques. While moving along the gradient is a good heuristic to improve the error value locally, it usually steers towards a local minimum. In the following we describe $L(\theta)$ as the objective function with parameters θ . Gradient descent (like all covered optimization algorithms) works iteratively. In each iteration we compute the velocity v i.e. the change of the parameter vector θ such that $\theta_{new} = \theta_{old} + v$ whereras $v = \beta v - \alpha \nabla L(\theta)$. For a first analysis we assume $\beta = 0$. As such the speed v is composed of the gradient ($\nabla L(\theta)$) and the learning rate α . Higher learning rate means we move further along the gradient in each step, but are therefore less flexible to intermediate changes in the objective function. However, if we set the learning rate too low, the algorithm takes a very long time to converge. Too high learning rate can lead to oscillation as seen in Figure 1b and Figure 1a, respectively.

To increase the speed of convergence we can use the dampening factor β . Successive movements in the same direction build up higher momentum

and allows SGD to take longer steps. The configuration in Figure 2d uses a low dampening factor. Compared to Figure 1b it converges in 74 steps instead of 216 steps, but slightly overshoots at the left turn due to the built up momentum. The effect of too high momentum is shown in Figure 1d. A method to alleviate this is the Nesterov momentum. Instead of computing the gradient at position θ , the gradient is computed at $\theta + v$. This can help because the gradient term always points in the correct direction, while the momentum term may not. This technique severely reduces spiralling around the optimum as shown in Figure 1e.

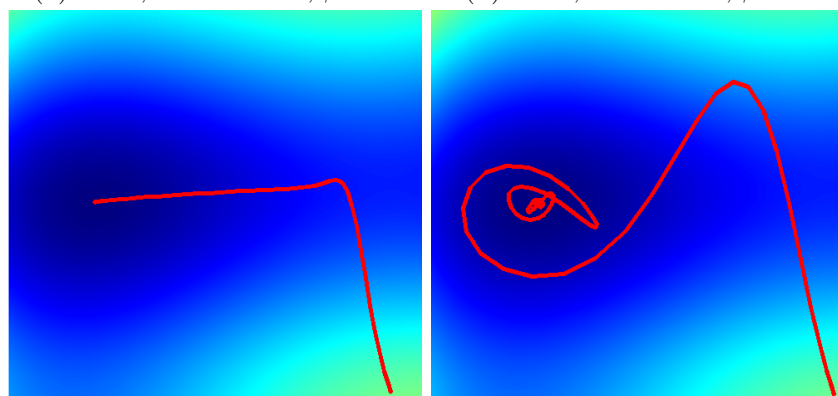
1.2 Comparison of Algorithms

Finally we compare the results of different algorithms on the same optimization problem. Most notably, we expect an optimization algorithm to converge to a minimum, and optimally of course the global minimum. As mentioned in the previous section, the strategy of following gradients is flawed as it steers the algorithm towards a local minimum. Secondly, the algorithm should show fast convergence towards the minimum. Figure 2 shows the results for SGD, RMSProp, Adam and AdamW (with default parameters according to PyTorch, for SGD lr=1) on the Eggholder function. Only AdamW (due to the by default high weight decay) is able to move past the local optimum to finally reach a better overall result. For this function it seems that only a different starting point would lead to better results for most algorithms. Figure 2 also nicely shows the behaviour near saddle points. At a saddle point the gradients become very low which considerably impacts convergence speed. To visualize this see the error over steps plotted in Figure 2e



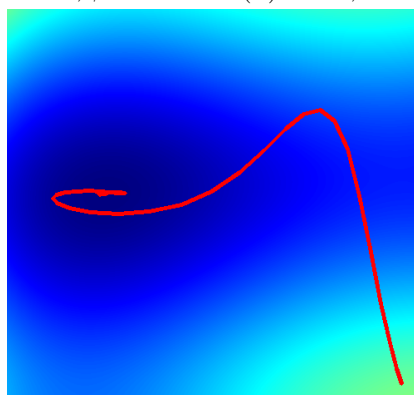
(a) SGD, $lr = 100000$, $\beta = 0$

(b) SGD, $lr = 10000$, $\beta = 0$



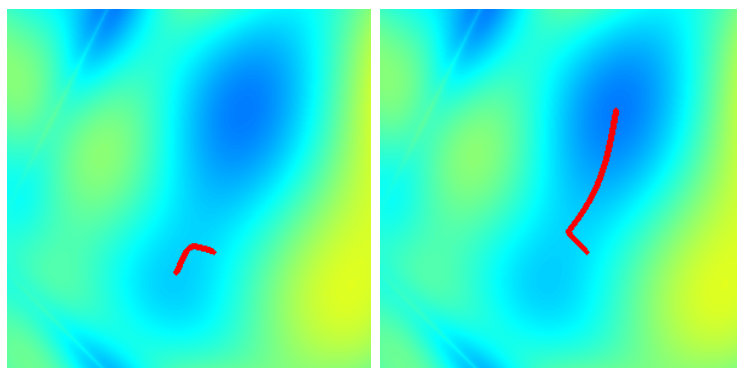
(c) SGD, $lr = 10000$, $\beta = 0.5$

(d) SGD, $lr = 10000$, $\beta = 0.9$



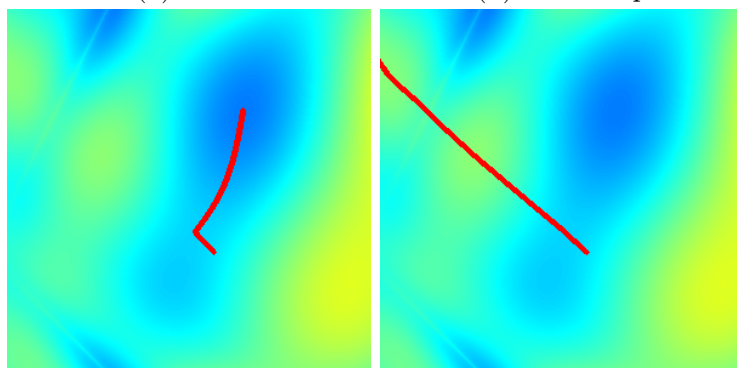
(e) SGD, $lr = 10000$, $\beta = 0.9$,
with Nesterov momentum

Figure 1: Shown are different configurations for SGD to emphasize the effect of different parameters.



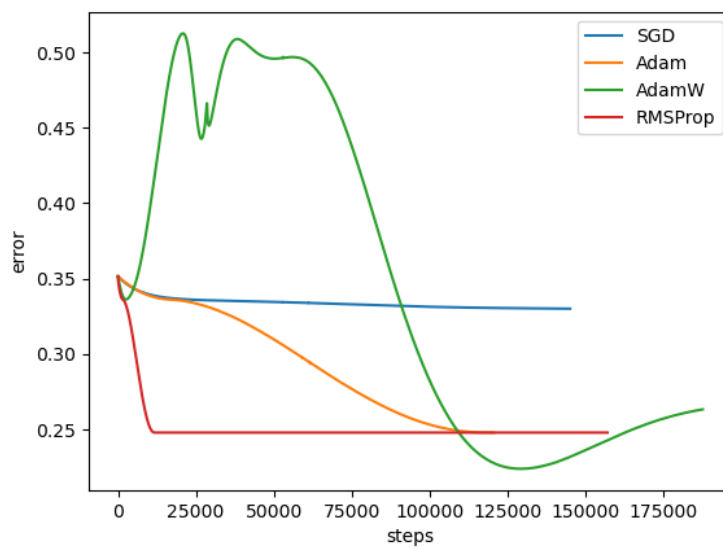
(a) SGD

(b) RMSProp



(c) Adam

(d) AdamW



(e) Loss over steps

4
Figure 2: Comparison of different optimizers